**Лабораторно упражнение 9**

**Създаващи шаблони. Шаблон „Строител“ (Builder)**

## I. Теоретична част

### 1. Шаблон „Строител“ - същност

Шаблон „Строител“ (Builder) се използва, когато е необходимо да бъдат създавани по-сложни обекти, състоящи се от повече елементи, като едновременно с това могат да бъдат представени в различни варианти.

#### 1.1. Цел и мотивация

Цел: Да се отдели конструирането на сложен обект от неговото представяне, така че в резултат на един и същ процес да се получат различни варианти на този сложен обект.

Мотивация:

Създаващият шаблон „Метод Фабрика“ скрива от клиентския код процеса на създаване на обектите, както и конкретния тип на връщания новосъздаден обект. Така шаблонът позволява развитието на йерархия от типове без да се засяга съществуващия клиентски код. С това се постига гъвкавост, но понякога отново е необходима, но от друг тип.

### 2. Пример за реализация на шаблон „Строител“

Шаблон „Строител“ ще бъде представен чрез следния пример: Да се създаде приложение, с което се представя превозно средство, което може да е лек автомобил или мотоцоклет.

Изграждането на приложението е чрез следните стъпки:

- Създава се клас, описващ продукт превозно средство

```
using System;
using System.Collections.Generic;
using System.Text;
namespace VehicleBuilder
{
    class Vehicle
    {
        private string vehicleType;
        private List<string> parts = new List<string>();

        public Vehicle(string type)
        {
            vehicleType = type;
        }
        public void Add(string part)
        {
            parts.Add(part);
        }
        public void Show()
        {
            Console.WriteLine(vehicleType);
            foreach (string part in parts)
            {
                Console.WriteLine(part);
            }
        }
    }
}
```

- Създава се абстрактен базов клас, съответстващ на „строител“

```
using System;
using System.Collections.Generic;
using System.Text;

namespace VehicleBuilder
{
    abstract class VehicleBuilder
    {
        protected Vehicle vehicle;
        public Vehicle Vehicle
        {
            get { return vehicle; }
        }
        public abstract void BuildFrame();
        public abstract void BuildEngine();
        public abstract void BuildWheels();
        public abstract void BuildDoors();
    }
}
```

- Създават се производните класове, които съответстват на конкретните строители

```
using System;
using System.Collections.Generic;
using System.Text;

namespace VehicleBuilder
{
    class CarBuilder : VehicleBuilder
    {
        public CarBuilder()
        {
            vehicle = new Vehicle("Car");
        }
        public override void BuildFrame()
        {
            vehicle.Add("CarFrame");
        }
        public override void BuildEngine()
        {
            vehicle.Add("Engine 2500 cc");
        }
        public  override void BuildWheels()
        {
            vehicle.Add("Wheels 4");
        }
        public override void BuildDoors()
        {
            vehicle.Add("Doors 4");
        }
    }
}
using System;
using System.Collections.Generic;
using System.Text;

namespace VehicleBuilder
{
    class MotorCycleBuilder:VehicleBuilder
    {
            public MotorCycleBuilder()
            {
```

```
                vehicle = new Vehicle("Motorcycle");
            }
            public override void BuildFrame()
            {
                vehicle.Add("Motorcycle Frame");
            }
            public override void BuildEngine()
            {
                vehicle.Add("Engine 500 cc");
            }
            public override void BuildWheels()
            {
                vehicle.Add("Wheels 2");
            }
            public override void BuildDoors()
            {
                vehicle.Add("Doors 0");
            }
        }
}
```

- Създава се клас директор

```
using System;
using System.Collections.Generic;
using System.Text;

namespace VehicleBuilder
{
    class Director
    {
        public void Construct(VehicleBuilder vehicleBuilder)
        {
            vehicleBuilder.BuildFrame();
            vehicleBuilder.BuildEngine();
            vehicleBuilder.BuildWheels();
            vehicleBuilder.BuildDoors();
        }
    }
}
```

- Създава се клиентският код

```
using System;
namespace VehicleBuilder
{
    class Program
    {
        static void Main(string[] args)
        {
            VehicleBuilder[] vehicleBuilders = new VehicleBuilder[2];
            Director producer = new Director();

            vehicleBuilders[0] = new CarBuilder();
            producer.Construct(vehicleBuilders[0]);
            vehicleBuilders[0].Vehicle.Show();
            Console.WriteLine();
            vehicleBuilders[1] = new MotorCycleBuilder();
            producer.Construct(vehicleBuilders[1]);
            vehicleBuilders[1].Vehicle.Show();
        }
    }
}
```

## II. Задачи за изпълнение

1. Да се допълни дадения примерен код като се добави информация за седалките на превозното средство. Да се включи в програмния код и обект автобус.

*Упътване: Към клас `VehicleBuilder` да се добави абстрактен метод, който добавя необходимия брой седалки на превозното средство. Да се добави и клас `BusBuilder`, който отговаря за създаване на обекта автобус.*

2. Да се приложение, което описва работата на павильон за сандвичи. Чрез приложението се създава обект „сандвич“, който съдържа информация за включените в сандвича продукти и неговата цена. Предлагат се два вида сандвичи – с месо и със сирене. За всеки един от двата вида могат да се използват бял или тъмен хляб. Към сандвичите се добавят до 3 вида салати. Сандвичите с месо могат да са със лук, краставици и домати, а сандвичите със сирене – с краставици, домати и салата айсберг. Цената и на двата вида сандвичи е фиксирана и не зависи от броя на салатите.

*Упътване: Класът на продукта е този, който описва обект „сандвич“. Интерфейсът (или абстрактен клас), който ще създава обекта да съдържа методи за: добавяне на хляб (като се посочва като параметър типа на хляба), добавяне на основния продукт, добавяне на салати, задаване на цена на сандвича. Този интерфейс се наследява от два класа, който отговарят за създаване на сандвич с месо и създаване на сандвич със сирене. Видовете салати могат да бъдат зададени като свойства от тип bool. Стойностите true и false определят дали е включена дадена салата в сандвича или не.*

*Класът, който съответства на „директор“ отговаря за това с какво да бъде изпълнен поръчания от клиента сандвич – с бял или тъмен хляб и с какви видове салати.*